

Proximity Server Real Time Nearby Discovery and Updates —

Production Design Date: 2025-08-10 UTC

Proximity Server Real Time Nearby Discovery and Updates —

Production Design Date: 2025-08-10 UTC

One Pager Summary Goal Deliver a global proximity service that keeps clients aware of entities near them and streams enter and leave and update events in real time. Target use cases include games proximity chat and events, AR and VR overlays, ride hailing, social presence, and IoT. The platform must be fast, reliable, privacy aware, and cost efficient.

Key Outcomes

Freshness under one second from device update to visible neighbor change. In region p95 end to end publish to notify under one hundred milliseconds for small fanout. Horizontal scale to millions of concurrent publishers and subscribers and billions of entities. Policy safe location handling with privacy controls and guardrails. Operational excellence with SLOs, autoscaling, canary and rollback, and deep observability.

Primary Assumptions

Entities represent users devices or assets that move and publish position and attributes. Updates arrive between one second and thirty seconds depending on client energy budgets. Fanout per entity ranges from a handful to thousands depending on density. Regions span Americas EMEA and APAC with optional data residency by tenant.

Functional Requirements

Core Upsert position with attributes and a stale after ttl. Query nearby within radius or k nearest. Subscribe to proximity topics to receive enter and leave and update deltas. Filters Attribute and relationship filters friend only same game instance same team access policy and block lists. Spaces Global earth coordinates and custom local spaces such as shards arenas or floors. Support height and two point five D. Rate and Budget Per tenant and per user rate limits quotas and density caps. Privacy Jittered locations or coarse cells for privacy preserving use, consent flags, and do not track zones. Admin Backfills, replay, policy updates, and dashboards.

Non

Availability ninety nine point nine five percent for read and write APIs and subscriptions. Latency targets p95 update ingest to notify under one hundred milliseconds in region and under one second cross region when replication is enabled. Durability RPO zero for accepted writes to the hot index and event log. RTO under fifteen minutes for a region. Scalability linear via shards and partitions. Cost aware autoscaling with density adaptive fanout. Compliance SOC two Type two, GDPR, and CCPA with minimization and retention windows.

Architecture Overview

Cell Based Each region runs a cell with Edge CDN, API and Websocket Gateways, Stream Ingest, Spatial Index, Match Engine, Fanout Workers, Hot Cache, Durable Store, and Observability. A thin global plane provides identity, configuration, routing, keys, and residency policy. Data Paths Write path client publishes position to Gateway which authenticates validates and enqueues to the streaming bus. Processor updates spatial index and computes neighbor deltas and pushes to fanout. Read path client issues nearby query which hits the spatial index. Subscribe path client subscribes to a proximity topic keyed by tenant and space and its own id or cell and receives deltas. Technology Stack Reference Edge and auth Cloud CDN WAF API Gateway OAuth two point one or OpenID Connect mTLS in mesh. Streaming Kafka or Pulsar with Schema Registry. Optional MQTT for device fleets bridged to Kafka. Processing Flink or Go services for update normalization and match engine. CRDT options for offline merges. Spatial index High performance geospatial service based on geohash or S two cells with per shard R trees for fine selection. Alternative PostGIS for small scale. Hot cache Redis Cluster for entity state, neighbor sets, and rate counters. Durable store DynamoDB Scylla or

Cassandra for entity state history and subscriptions. Object storage for cold archives Parquet with Iceberg or Delta. Pub or sub Gateways maintain Websocket or QUIC sessions and subscribe to per user topics.

Optional NATS for intra cell pub or sub. Platform Kubernetes service mesh with mutual TLS. Argo CD and Argo Rollouts. Terraform or Pulumi and Helm.

Data Flow Ingest Client signs and sends UpdatePosition with entity id tenant id space id coordinates timestamp velocity and attributes. Gateway authenticates and rate limits and writes to Kafka topic partitioned by tenant and cell id. Deduplication uses event id and short ttl. Process Update normalizer snaps to cell grid computes geohash keys and writes entity state to Redis and durable store. Match engine queries neighbor candidates using cell rings applies attribute filters and hysteresis then emits enter leave and update deltas. Fanout Workers publish deltas to per subscriber channels. Websocket Gateway pushes frames to connected clients. Offline subscribers receive a since token for catch up on reconnect. Query Nearby query hits spatial index with radius or k nearest and attribute filters optionally limited to max results and density caps.

Storage Strategy

Hot Redis hashes keyed by tenant id and entity id for positions and attributes, sorted sets per cell for entity lookup by time, and per subscriber neighbor sets with short ttl. Use hash tags to collocate related keys. Warm NoSQL store for authoritative entity states and subscription specs with time ordered history optional for audit. Cold Object storage for daily partitions of cell density snapshots and delta streams in Parquet. Analytics warehouse ClickHouse or BigQuery for heatmaps and flows. Scaling Approach Sharding Space is divided into hierarchical cells by geohash or S two level. Shards own ranges of cell ids. Consistent hashing maps entities to shards. Hot cells can be split by increasing precision or adding a suffix shard id. Load control Density aware fanout clamps per tick neighbor counts and prioritizes closer entities. Coalesce frequent updates with deadbands on distance and heading. Use leaky bucket per subscriber to avoid backpressure collapse. Cross region Default no cross region fanout. Optional asynchronous replication for global experiences with higher budget. Residency policy prevents illegal movement of location data.

Data Models

EntityState tenant id, space id, entity id, cell id, lat, lon, z, velocity, heading, attributes json, stale at, updated at. Subscription tenant id, entity id or group id, space id, filter spec, last token, created at. DeltaEvent tenant id, space id, subscriber id, type enter or leave or update, subject id, position snapshot, attributes diff, sequence, created at. Cell space id, cell id, bounds, density metrics, last compaction at. Policy tenant id, privacy mode, jitter meters, min cell size, retention days, block zones, created at.

Latency Budgets and SLAs

Gateway processing under fifteen milliseconds p95. Produce to stream under fifteen milliseconds p95. Processor update and match under twenty milliseconds p95. Redis read or write under three milliseconds same AZ p95. Fanout enqueue under ten milliseconds p95. End to end in region under one hundred milliseconds p95. Availability ninety nine point nine five percent per region for APIs and subscriptions.

Key Algorithms and Workflows

Spatial partitioning Geohash or S two mapping to fixed level cells. Candidate search expands to ring of neighbor cells for radius. For k nearest perform incremental cell expansion with heap on distance squared. Hysteresis and flapping control Enter and leave require position change greater than thresholds on distance and time. Update coalescing uses deadbands and velocity based prediction. Privacy preservation Optional jitter adds small random offset within a configured radius on client and server. Coarse mode reports cell id only. Block zones deny fanout inside sensitive areas. Crowd control When density exceeds cap, choose closest N by distance and priority using a bounded heap. Remaining entities are sampled to avoid starvation. Offline tolerant Client can queue recent positions with lamport sequence. Server merges using last write wins per entity id and timestamp within a guard band. Abuse mitigation Per ip and per token limits, anomaly detectors for teleport and impossible

velocity, signed updates with device attestations when available.

Security Reliability and Compliance Security TLS one point three in transit, KMS at rest, OAuth scopes and RBAC, mTLS service to service, signed webhooks. Input validation, coordinate sanity checks, and SSRF or egress allow lists for any callbacks. Privacy Minimize PII, pseudonymous ids, residency by tenant and space, consent flags, retention controls, and export or delete APIs. Reliability Multi AZ deployments, replay via Kafka, DLQs and quarantine queues, graceful degradation to cached neighbors, disaster recovery with tested restores. Change management Canary rollouts with automatic rollback on SLO burn, feature flags for filters, schema registry for evolution.

Multi Tenancy Tenant id carried in every key, topic, and metric. Quotas and budgets per tenant. Soft isolation through namespaces and partitions. Optional hard isolation with dedicated cells and VPCs and per tenant keys.

CI CD and Infrastructure as Code Everything as code with Terraform or Pulumi and Helm. Argo CD and Argo Rollouts enable canaries and rollbacks. Contract tests for schemas. Static analysis and security scans. Online migrations and dual write or shadow read windows for breaking changes.

APIs and Integration Points Client POST update position, GET or POST subscribe proximity, GET nearby, POST unsubscribe, Websocket subscribe to delta topic, GET since for catch up. Server integrations Signed service messages to trigger system events and to moderate. Webhooks for density alerts and zone updates. Batch export to analytics and data lake. Telemetry QoS probes and synthetic clients per region.

Observability and Operations

Golden signals Ingest lag, match time, p95 and p99 end to end latency, Redis ops, fanout queue depth, dropped delta rate, density caps triggered, hot cell count, error budget burn. Logs and traces Structured logs with tenant id, space id, cell id, entity id, and sequence. Traces from gateway to processor to fanout to client ack. Runbooks Cold start and cache warm, shard split or merge, hot shard mitigation, stream backlog drain, regional failover. Autoscaling Based on lag, p95 latency, shard CPU, and hot cell counts. Pre warm capacity for events.

Capacity Planning and Cost Estimate Illustrative per busy region. Ten million daily active entities and two hundred thousand updates per second sustained with peaks up to one million. Kafka six to nine brokers replication factor three. Redis twelve to twenty four shards. Processing pool sized to partition count with checkpoint intervals under five seconds. NoSQL provisioned for write heavy workloads. Rough monthly cost range thirty thousand to ninety thousand depending on density and retention.

Testing Strategy and Failure Drills

Unit and property tests for spatial queries and hysteresis. Contract tests for update schemas. Load tests with skewed density hot cells and bursty movement. Soak tests for memory and index fragmentation. Chaos drills for broker loss shard failover and regional isolation. Disaster recovery drills for durable store and object storage.

Phased Build Plan

MVP three months Single region cell with update and nearby query basic subscriptions Redis hot index Kafka ingest NoSQL durable state and minimal console. Phase two six to nine months Multi region cells presence or typing style deltas richer filters density aware fanout analytics taps canary delivery and rollback. Phase three twelve months Cross region options hard tenant isolation advanced privacy modes hot cell auto split and cost optimizations with adaptive sampling.

Key Choices Trade Offs Risks and Mitigations Cell based spatial index gives predictable latency but requires careful split and merge logic. Mitigate with automation and metrics. Fanout on write delivers immediacy but can be expensive in very dense areas. Hybrid fanout and sampling reduce cost. Coarse privacy lowers utility for precision apps. Offer tenant selectable modes and explicit consent. Streaming backbones add operational cost but provide replay and isolation. Use managed services.